

(Title Slide)

Good morning everyone, my name is Manuella Ehab, and this is a presentation about my bachelor thesis topic titled: 'GO-DRiVeS Mobile App for Passenger-Vehicle Real-Time Interaction with In-Campus Golf Car.'

(Outline Slide)

In today's presentation, I will briefly introduce my topic, the problem we are trying to solve, and then discuss the literary review conducted and the research gap we are covering. Afterwards, I will talk about my project objectives, methodology, and results so far. And finally, I will conclude the presentation and give future recommendations.

(Introduction - Slides 1 & 2)

So, what are autonomous vehicles? They're self-driving cars, cars that don't need a human to control them. You've probably heard of Tesla, right? Elon Musk's company? That's one of the big names pushing this technology forward. It's honestly a game-changer for transportation.

Now, why do AVs make sense on a campus? Well, campuses are usually big, and walking from one end to another can take time. With AVs, students can get to class quicker, especially when they're running late. They also help when the weather is bad, like windy winter days or extreme heat, so students don't have to walk far. And because they can be tracked and managed, they make transportation safer and more organized.

And that's exactly what my project, *GO-DRiVeS*, is built to solve. It's a mobile app made to simplify campus transportation by connecting students and staff with autonomous golf cars. The key focus is enabling *real-time interaction* between passengers and the AVs—so users can track the vehicle, request rides, and stay updated instantly. For this project, I focused mainly on developing the passenger side of the app.

(Problem)

Currently, there's a lack of good solutions for real-time interaction between users and autonomous vehicles on campus. Existing ride-hailing

apps like Uber or Kareem are not designed to handle AVs. While AV tracking has been researched, it hasn't been implemented in a user-friendly mobile app. GO-DRiVeS fills this gap by providing a dedicated platform for seamless AV interaction.

(Literature Review - Slides 3 & 4)

In my literature review, I examined prior research on real-time transport and AV technologies, particularly within campus environments. Much of the existing work focuses on vehicle-centric aspects like sensor fusion and path planning, with less emphasis on the user interface. Many studies also lack real-time, mobile-based interaction between users and AVs, especially in closed campus settings.

For example, Narasimhan et al. (2021) developed a bus tracking app using Firebase and Google Maps API, but it only tracked the bus's location using the drivers phone, there was no passenger-vehicle interaction. Similarly, Sharif et al. (2018) created a real-time campus bus tracking system with GPS modules and a Firebase backend, but it didn't include a user-vehicle interaction layer or mobile-side logic either. Niture et al. (2021) proposed an IoT-based smart vehicle system with accident alerts and ZigBee communication, but it involved a complex hardware setup and wasn't mobile-first. Yang & Kuo (2021) worked on an AV prototype with multisensor fusion and reinforcement learning for tracking, which was campus-specific but focused on the vehicle positioning and path planning rather than the user interface.

When reviewing similar projects from the past decade, I identified common technology choices. Many developers used Firebase for backend and database management, but I went with MongoDB to create my own custom database and to avoid Firebase's pricing limitations. For mobile app development, tools like Blynk, Flutter, and MIT App Inventor were popular for their simplicity, but I chose React Native with Expo to have greater control and faster development, as i had previous experience with them. While Google Maps is widely used, I found OpenStreetMap to be a suitable and free alternative with an accurate map of our GUC campus.

(Research Gap)

While many ride-hailing apps like Uber or Lyft exist, they aren't built for autonomous vehicles or campus environments. Existing AV apps often focus more on the vehicle's performance and less on user interaction. There's also limited research or implementation of real-time, user-facing mobile apps that connect passengers directly with autonomous vehicles, especially for short-distance, in-campus transport like golf cars. GO-DRiVeS fills this gap by creating a dedicated mobile app that enables **real-time interaction** between passengers and AVs, specifically tailored for a campus setting.

(Objectives)

The primary objectives of this project were to:

- Design a simple mobile interface for passengers to interact with the AV and track its data.
- Display live camera feeds with bounding boxes on detected pedestrians, vehicles, and scene objects.
- Provide passengers with essential journey information, like the location, speed, and ETA (estimated time of arrival).
- Alert users about important events during the ride, such as pedestrian crossings.

(Methodology)

The methodology for this project was divided into four main phases:

first step was planning where collected information to understand the project and what users need.

Researched similar apps and AV systems to get ideas of what's possible, and research gaps.

Started planning what basic features are needed

- **Designing (Week 2):** This phase focused on creating UI/UX designs for both passenger and operator interfaces, eliminating unnecessary features and developing a data flow diagram to illustrate the app's interaction with the AV system. a dataflow diagram of all the subsystems. (included diagram and explain the nodes)

- **Development (Weeks 3-5):** This was the main implementation phase, where I built the mobile app using React Native with Expo, developed the backend with Node.js and Express, set up the MongoDB database, and integrated ROS2 for communication with the autonomous golf car. Key development milestones included publishing the car's location and speed via ROS and listening for data related to passengers, vehicles, and objects.
- **Testing (Week 6):** The final phase involved testing the app using Expo Go to verify location and speed tracking accuracy, testing the backend deployment, and ensuring proper data exchange through ROS nodes.

Throughout the development process, I encountered several challenges, including the complexity of integrating ROS2 with a mobile application and ensuring real-time data synchronization.

(Further Testing)

To further validate the app's functionality, I've outlined several additional tests:

- **Route Changing:** Testing how the app notifies passengers and displays route changes.
- **Network Disconnection Test:** Evaluating the app's behavior when the network connection is lost.
- **Location Accuracy Test:** assess the precision of location tracking using OpenStreetMap while the vehicle is moving.
- **Speed Update Delay Test:** Measuring any delay between the vehicle's actual speed and its display in the app.

(Results)

The results of this project demonstrate the successful development of a working mobile app for real-time interaction between passengers and in-campus autonomous vehicles. Key achievements include:

- Accurate display of real-time vehicle location and speed using device GPS.
- Successful integration of ROS2 for data exchange with the AV system.
- Development of a custom backend and authentication system.

(Conclusion)

To wrap up, the GO-DRiVeS app has achieved its core objectives by connecting users with self-driving golf cars through a user-friendly mobile interface. It provides essential features like live tracking and dedicated interfaces for both passengers and pedestrians. This project represents a significant step towards modernizing and enhancing campus transportation. With further testing and the addition of new features, GO-DRiVeS has the potential to become a valuable tool for improving daily mobility on campus.

(Future Recommendations)

To further enhance GO-DRiVeS, I recommend the following:

- Implementing offline mode with auto-sync functionality to handle network interruptions.
- Adding accessibility features like larger buttons and voice feedback for visually impaired users.